TASK 2 REPORT

Algorithms & Problem Solving

Merge Sort, Binary Search, and Interval Scheduling

Student Name	Sohail Khan
Internship	C / C++ Programming Internship
Task Number	2

This report explains the task goal, design, implementation plan, sample test cases, complexity, and steps for building and submitting the project on GitHub.

1. Objective

Solve at least three algorithmic problems involving sorting, searching, and greedy/dynamic problem-solving. This report describes three suitable C++ implementations: **Merge Sort**, **Binary Search**, and **Interval Scheduling**.

- Provide source files and test inputs.
- Test multiple input sizes and record execution time.
- Explain the algorithm choice and time/space complexity.

2. Problem A — Merge Sort (Sorting)

Merge Sort uses divide and conquer. It splits an array into halves, sorts each half recursively, and merges the sorted halves into one sorted array.

Steps

- If the array contains zero or one item, it is already sorted.
- Divide the array into left and right halves.
- Sort both halves recursively, then merge them in ascending order.

Example

```
Input:  38 27 43 3 9 82 10\nOutput: 3 9 10 27 38 43 82
```

Time complexity is O(n log n) in best, average, and worst cases. Auxiliary space is O(n). Merge Sort is useful when predictable performance is more important than in-place sorting.

3. Problem B — Binary Search (Searching)

Binary Search finds a target in a **sorted** array by repeatedly checking the middle element and discarding the half that cannot contain the target.

Steps

- Set low to the first index and high to the last index.
- Calculate mid, compare the middle value with the target, and keep the appropriate half.
- Stop when the target is found or low becomes greater than high.

Example

```
Sorted input:  2 5 8 12 16 23 38\nTarget: 16\nOutput: Found at index 4 (zero-based)
```

Time complexity is O(log n); iterative binary search uses O(1) extra space. It must not be used on unsorted input unless the data is sorted first.

4. Problem C — Interval Scheduling (Greedy)

Given activities with start and finish times, select the maximum number of non-overlapping activities. The greedy strategy sorts activities by finish time and always selects the next compatible activity that finishes earliest.

Steps

- Sort all activities by ascending finish time.
- Select the first activity.
- For each later activity, select it if its start time is greater than or equal to the finish time of the last selected activity.

Example

```
Activities (start, finish): (1,3), (2,5), (4,7), (6,9), (8,10)\nOne valid selection: (1,3), (4,7), (8,10)
```

Sorting takes O(n log n), and the selection pass takes O(n), so total time is O(n log n). The greedy rule is optimal for the standard unweighted interval-scheduling problem.

5. Test Plan and Runtime Measurement

Use the same compiler and computer for each timing run. Test increasing input sizes, such as 1,000, 5,000, 10,000, and 20,000 items. Run each test several times and record the average; avoid timing input/output if possible.

```
Example test matrix\nAlgorithm        Small input       Larger input       Correctness check\nMerge Sort       10 va
```

Do not invent benchmark numbers in the report. Fill in the measured runtime table after running your own program.

6. Build and Run

Keep each algorithm in a separate file, or use one menu-driven program. Example commands for separate files:

```
g++ -std=c++17 -O2 -Wall -Wextra merge_sort.cpp -o merge_sort\n./merge_sort\n\ng++ -std=c++17 -O2 -Wall -Wextra binar
```

Use input files or fixed test cases to make results repeatable. Include both successful and unsuccessful search cases and edge cases such as an empty list.

7. Suggested GitHub Structure and Conclusion

```
task-2-algorithms/\n■■■ merge_sort.cpp\n■■■ binary_search.cpp\n■■■ interval_scheduling.cpp\n■■■ test-inputs/\n■■■
```

These three problems cover sorting, searching, and greedy optimization. Before submission, compile and test each implementation, add actual runtime measurements and terminal screenshots, and ensure the README gives exact build commands.